



Figure 7: Gradient descent



Figure 8: Gradient descent on an uneven surface

you can see, this is more than enough information to find the bottom of the bucket in a few iterations. This is a sub-field of optimisation called gradient optimisation (gradient is just a fancy word for slope or steepness).

Note 4: To know more, you can refer to the article ‘Improving our neural network by optimising Gradient Descent’, posted by lamtrask, better known as ‘Trask’.

The complexity of calculating the slope will become more and more challenging if, instead of a bucket, we have an uneven surface (like the surface of the moon or Mars). Getting the ball to the lowest position in this situation needs a lot of mathematical calculations and also more data sets.

Gradient descent isn't perfect. There are some solutions around the corner that can help us to overcome these challenges, like:

- Using multiple random starting states
- Increasing the number of possible slopes and considering more neural networks
- Using optimisations like ‘Native Gradient Descent’
- Adding and tuning the alpha parameter

Note 5: What is alpha? As described above, the alpha parameter reduces the size of each iteration's update in the simplest way possible.

Finally, gradient descent that uses non-linearities to solve the above problem is popular. Here, Sigmoidal neurons are used as training neurons.

The back-propagation algorithm: The back-propagation approach was pioneered by David E., Geoffrey Hinton and Ronald J. Sometimes we do not know what the hidden units are doing, but what we can do is compute how fast the error function changes as we change the hidden activity. With this we can find out how fast the error changes when we change the weight of an individual connection. Using this, we try to find the steepest descent.

Each hidden unit can affect many output units, and to compute the error derivatives for the activities of the layers below, we have to back-propagate to find the nearest values;

hence, the name.

Stochastic and mini-batch gradient descent: There are three variants of gradient descent, which differ in how much data is used to compute the gradient of the objective function. Depending on the amount of data, one has to make a trade-off between the accuracy of the parameter update and the time it takes to perform an update. Instead of a single static error surface, by using a dynamic error surface and by descending on this stochastic surface, the ability to navigate flat regions improves significantly.

Note 6: Do refer to ‘An overview of gradient descent optimisation algorithms’ by Sebastian Ruder.

- Batch gradient descent:** Batch gradient descent (also known as Vanilla gradient descent) computes the gradient of the cost function with respect to the parameter θ for the entire training data set:

$$\theta = \theta - \eta \nabla J(\theta)$$

- Stochastic gradient descent:** Stochastic gradient descent (SGD), in contrast, performs a parameter update for each training example — $x(i)$ and label $y(i)$:

$$\theta = \theta - \eta \nabla J(\theta; x(i); y(i))$$

Batch gradient descent performs redundant computations for large data sets, as it recomputes gradients for similar examples before each parameter update. SGD does away with this redundancy by performing one update at a time. It is therefore usually much faster and can also be used to learn online.

- SGD fluctuation:** Batch gradient descent converges to the minimum point of the basin the parameters are placed in. On the one hand, SGD's fluctuation enables it to jump to new and potentially better local minima, but on the other hand, this complicates convergence to the exact minimum, as SGD will keep overshooting. However, it has been shown that when we slowly decrease the learning rate, SGD shows the same convergence behaviour as batch gradient descent, almost certainly converging to a local or the global minimum for non-convex and convex optimisation, respectively.

$$\theta = \theta - \eta \nabla J(\theta; x(i:n); y(i:n))$$

Note 7: Machine learning systems are mainly classified as supervised/unsupervised learning, semi-supervised learning, reinforcement learning, batch and online learning, etc. All these systems mainly work on training the data set. These broad categories are sub-divided into respective algorithms based on the training data set. A few popular examples are: k-nearest neighbours, linear and logistic regression, SVMs, decision trees and random forests, neural networks (RNN, CNN, ANN), clustering (k-means, HCA), etc.

Continued to page 95...